FRED TALKS

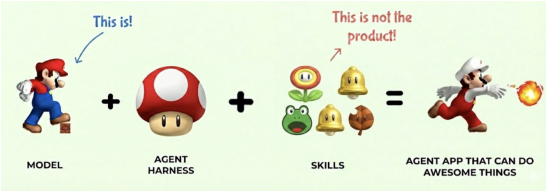
15th June 2026

CONTENTS

It's-a Me, Agentic AI

HAN, NOT SOLO · 01 MAR 2026 · SOURCE

Agentic AI is a fairly recent development that combines reasoning (OpenAI, 2024) tool use (Schick et al, 2023) in the same AI model. But an agentic AI system is not just the model, but also the harness, the environments, the tools, rewards, evaluations, and all of the infrastructure to support it. In this post, let's use the top grossing franchise, Mario, to tell this story for understanding how agentic AI models are developed, how agent harnesses work, and how reinforcement learning ties everything together. If you survived World 8-4 as a kid, you already have the intuition for building agentic AI systems.



alt text

Small Mario is the **base pretrained model**. He's just come out of pretraining on a massive corpus of platform game physics. He can walk. He can jump. He can move left and right. These are his base capabilities, the raw knowledge compressed from the training data.

But Small Mario is fragile. One hit from a Goomba and he's dead. He can't break bricks. He can't take damage. He has potential, but he's not yet useful for anything beyond the most trivial tasks.

This is your base LLM fresh off pretraining. It has absorbed enormous amounts of knowledge, it can do next-token prediction, and it can sort-of follow instructions. But ask it to do anything real, reliably, in production, and it falls apart on the first obstacle. One Goomba and it's game over.

The Super Mushroom: Model Harness

Then Mario finds the **Super Mushroom**. He doubles in size. He can now break bricks. He can take a hit without dying. He goes from fragile to capable.

References

- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS. https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html

- Lee, H. (2026). A Taxonomy of RL Environments for LLM Agents. Han, Not Solo. https://leehanchung.github.io/blogs/2026/03/21/rl-environments-for-llm-agents/

- Workman, G. (2026). How Ramp built a full-context background coding agent on Modal. Modal Blog. https://modal.com/blog/how-ramp-built-a-full-context-background-coding-agent-on-modal

- Lopopolo, R. (2026). Harness engineering: leveraging Codex in an agent-first world. OpenAI. https://openai.com/index/harness-engineering/

- Microsoft. (2024). Azure Container Apps Dynamic Sessions. https://learn.microsoft.com/en-us/azure/container-apps/sessions

- Firecracker. AWS open-source microVM. https://firecracker-microvm.github.io/

- Northflank. (2026). Top Modal Sandboxes Alternatives for Secure AI Code Execution. https://northflank.com/blog/top-modal-sandboxes-alternatives-for-secure-ai-code-execution


```
@article{
    leehanchung,
    author = {Lee, Hanchung},
    title = {Hidden Technical Debt of AI Systems: Agent Runtime},
    year = {2026},
    month = {04},
    day = {24},
    howpublished = {\url{https://leehanchung.github.io}},
    url = {https://leehanchung.github.io/blogs/2026/04/24/hidden-tech
}
```

There are three honest paths through this:

Co-locate train and prod on the same runtime. Pick one sandbox provider, run RL rollouts on it, run production sessions on it, accept the lock-in. This is what the most disciplined AI engineering teams are doing.

Define a runtime contract and enforce it on both sides. A small, versioned interface — "this is the shell, these are the tools, these are the latencies, these are the failure modes" — that you implement once on your training infrastructure and once on your production infrastructure. The contract is the abstraction; the sandbox underneath can vary. This is harder than it sounds because the contract has to cover not just the API surface but the timing and failure semantics that the agent has learned.

Train against production noise. Inject latency, errors, and tool failures during training so the agent's policy is robust to runtime variance instead of dependent on a specific runtime. Step-DeepResearch reports tangible gains from injecting 5–10% tool errors during training. This is the runtime analog of dropout.

The wrong answer, the one most teams are unconsciously picking, is to pick a sandbox for production as a software engineering solution, and forget all about the requirements from AI and machine learning, then spend the next quarters chasing flakiness of agent performances.

The Bill That Is Coming Due

Sculley's argument was that ML systems accumulate technical debt in places that traditional software engineering does not have language for — feedback loops, configuration sprawl, undeclared consumers, glue code. Agent systems inherit all of that and add a new line item: the runtime debt.

Runtime debt looks like this. A team picks the sandbox that was easiest to integrate at prototype time. Production traffic exposes a different mix of failure modes. The team patches around the gap with retries, longer timeouts, and prompt-engineered apologies. The agent's behavior gets entangled with that runtime's quirks with whack-a-mole without disciplined research and development. A year later, switching runtimes is a six-month migration because nobody can predict which behaviors will move.

The agent has to live on a runtime. The teams that internalize that early will spend the next two years compounding the advantage. The teams that do not will spend the next two years paying down a debt they did not know they were taking on.

The Super Mushroom is the **model harness**, the entire infrastructure and post-training pipeline that transforms a base model into something production-ready. This includes:

- **Instruction tuning** (RLHF, DPO, etc.) to make the model actually follow directions
- **System prompts** that define personality and constraints
- **Safety guardrails** so it doesn't run off cliffs
- **Memory and context management** so it remembers where it's been
- **Tool-use training** so it knows power-ups exist and how to grab them

Without the Super Mushroom, Mario is a liability. With it, he's a platform for greatness. Similarly, without the model harness, a base LLM is a research artifact. With it, it's a product.

> The Super Mushroom doesn't change WHO Mario is. It changes what he can SURVIVE. The model harness doesn't change the model's core knowledge. It changes what the model can handle in production.

Now here's where it gets interesting. Super Mario can pick up **power-ups** that give him entirely new capabilities:

Power-Up	Mario Ability	Agent Equivalent
Fire Flower	Throw fireballs to defeat enemies at range	**Code execution** - reach out and run code to solve problems the model can't solve with text alone
Frog Suit	Swim through water levels with ease	**Web search** - navigate and retrieve information from an environment the model wasn't trained on
Raccoon Leaf	Fly and reach otherwise inaccessible areas	**File system access** - explore and modify the codebase, reach files the model couldn't otherwise touch
Hammer Suit	Throw hammers, defeat almost anything	**Shell/terminal access** - the most powerful tool, can interact with the full system
Star	Temporary invincibility, blow through everything	**Extended thinking/reasoning mode** - brute force through complex problems at higher compute cost
Cape Feather	Sustained flight and spin attack	**MCP servers** - sustained, extensible access to external services and APIs

Each power-up doesn't replace Mario's core abilities. Mario still walks and jumps. The power-ups **extend** what he can do. A Fire Flower Mario can still jump on Goombas, but now he can also shoot fireballs at Piranha Plants hiding in pipes.

This is exactly how agent tools work. The LLM still does what LLMs do: reasoning, language understanding, and planning. Tools extend the model's reach into environments it can't operate in alone. An LLM can't execute Python by itself, just like Mario can't throw fireballs without a Fire Flower. But give it the right tool, and suddenly the problem space opens up.

And critically, **Mario has to learn WHEN to use each power-up**. Frog Suit is amazing in water levels, useless on land. Fire Flower is great against Goombas, pointless against Thwomps. The model needs to learn tool selection, knowing which tool to reach for in which context. This is one of the hardest parts of building agentic systems.

One power-up deserves special attention: the **Star**. When Mario grabs a Star, he becomes invincible. He plows through Goombas, Koopa Troopas, Piranha Plants, everything in his path just disintegrates. Nothing can stop him. This is not dissimilar to having an engineering manager who's really good at clearing organizational blockers for their engineers. The Goombas and Piranha Plants of bureaucracy, cross-team dependencies, access requests, and priority conflicts just melt away. Star power is temporary and expensive, but when you need to blast through a critical path, nothing else comes close.

The Mushroom Kingdom: Environments and Tasks

Now let's talk about the world Mario operates in. Every level in the Mushroom Kingdom is an **environment**, and every environment is composed of the same building blocks:

Level Element	Environment Equivalent
Bricks	Structured data, APIs, databases that can be broken open to reveal resources
? Blocks	Unknown information sources, sometimes containing exactly what you need
Pipes	Entry points to sub-tasks, function calls, or deeper exploration
Coins	Incremental progress signals, intermediate rewards
Goombas	Common obstacles, predictable errors, edge cases
Koopa Troopas	Recyclable obstacles, errors whose solutions can be reused as tools
Piranha Plants	Traps hiding in seemingly safe passages, hallucination triggers
Vines	Hidden paths to bonus areas, unexpected shortcuts

A training rollout wants to start in 200 milliseconds, run for thirty seconds against a frozen snapshot of the world, get scored, and disappear. A production session wants to start in two seconds, hold state for a forty-minute conversation, talk to the live internet, and survive a transient failure without losing the user's work.

The production side has a requirement that almost nobody surfaces in eval reports: the agent needs to survive the **async gaps** in real engineering work. Cognition's experience report is the cleanest articulation of this — an agent opens a PR, waits on CI, responds to a review comment, reruns tests, pushes a follow-up commit. Between each step there are minutes, hours, sometimes days where the agent's working state has to persist. A containerized agent can only survive these gaps by burning compute to stay alive, and if the container is rescheduled or times out the session is lost. Their solution was hypervisor-level snapshotting of the full machine state — memory, process tree, filesystem — so compute shuts down while the agent is idle and resumes exactly where it left off when a CI result arrives. Building that took longer than any other piece of infrastructure they had shipped to date.

Optimizing the same runtime for both training and production is how you end up with a system that is too slow for training and too brittle for production. Most teams discover this after the first time they try to "just deploy what we trained on" and find the latency budget eaten by a startup model that was fine when amortized over ten thousand rollouts and intolerable when paid by a single user staring at a spinner.

Dev/Prod Parity Is the Real Problem

In 2011, Twelve-Factor App told us to keep development, staging, and production as similar as possible. That advice was about reducing the gap between where the engineer types code and where the code runs. For agents, the gap that matters is between where the agent **trained** and **experimented** and where the agent **runs**.

An agent learns the runtime. Tool latencies, failure modes, shell quirks, filesystem layout, the exact way `ls` formats output, the way the browser resolves redirects. The model picks up all of it during training and bakes it into the policy, or the optimizer (RLM or GEPA) picks up all of it during training and bakes it into the harness or the prompts. Move these to a different runtime and the behavior shifts. Tools that were idempotent become flaky. Commands that were instant now block. Snapshots that existed disappear. The agent has no abstract concept of "shell"; it has a concept of "the shell I trained on."

This is a new flavor of distributional shift. It is not the data shift the MLOps community has been worrying about for a decade. It is **runtime shift**, and it shows up as silent quality regressions that no eval catches because the eval runs in the training runtime.

Lambda, Fargate, App Runner, Cloud Run. It is tempting to use these as agent sandboxes because they are cheap and already in your account. They are designed for stateless, request-response workloads with strict execution-time limits and no native snapshot model. They will work for a demo. They will not work for an agent that runs for six hours, mutates a filesystem, and needs to fork mid-trajectory.

The hyperscaler offerings are converging on the right shape — microVM isolation, fast snapshots, managed tools — but the primitives underneath were built for web workloads. The startups had the advantage of building on top of those primitives without inheriting the legacy product surface area.

Experimentation and Production Want Different Runtimes

One of the major difference how AI engineering differes from traditional web development is the difference in development and production infrastructure requiremetns. Here's the difference between your training cluster (if you are training models), experimentation and evaluation runs, vs production workload.

Dimension	Experimentation / Training	Production / Serving
Concurrency shape	Thousands of parallel rollouts, bursty	One session per user, steady-state
Cold start tolerance	Critical — 5s × 10k rollouts is real money	Forgivable — users wait
State model	Fork, branch, replay, snapshot	Durable, per-user, auditable
Network policy	Often offline or recorded for determinism	Open internet, real APIs
Failure model	Drop the rollout, sample more	Retry, degrade, page someone
Observability	Full trace, every token, replayable	SLOs, error budgets, sampling
Cost model	Spot, preemptible, batch	Reserved, predictable, latency-bound
Determinism	Required for reproducible runs	Often counterproductive
Lifetime	Seconds to minutes	Minutes to hours, or pinned for a session
Clouds / Platforms	Abstractions and scaffolding that support traversal	
Pits	Catastrophic failures, unrecoverable errors	

Every level remixes these elements differently. World 1-1 is simple, a few Goombas, some bricks, a clear path to the flag. World 8-4 is a maze of pipes, hidden paths, and a boss fight with Bowser. Same building blocks, radically different difficulty.

Throughout each level, Mario interacts with the environment as **tool use**. He enters pipes to warp from one place to another, the equivalent of an API call that transports you to an entirely different context. He bumps ? Blocks from below to discover power-ups, new agent skills materializing from the environment when you know where to look. He breaks bricks to clear paths or reveal hidden rewards, structured data yielding its value when you apply force in the right direction. He stomps on a Koopa shell and kicks it forward, turning an obstacle into a projectile that clears a line of Goombas, repurposing error outputs as inputs to solve downstream problems. The environment isn't just a backdrop. It's a toolbox.

Having an environment is not enough. We need to define **what we want to achieve** from playing the game. These are the **tasks**.

Mario can play the same level with completely different objectives. Complete the level. Complete the level in the shortest amount of time. Get the highest score. Collect the most coins. Accumulate the most 1-up lives. Find all the hidden rewards. Stomp on every last Goomba. Each objective produces a fundamentally different play style from the same environment.

This is exactly the task definition problem in agentic AI. The same model operating in the same environment can be pointed at wildly different goals. "Summarize this codebase" and "refactor this codebase" use the same files, the same tools, the same context, but they require entirely different strategies. The task is what transforms an environment from a sandbox into a mission.

The Flagpole: Evaluations and Reward Modeling

At the end of every level, there's a **flagpole**. Mario jumps on it, pulls down the flag, and receives a reward. The higher he grabs the flag, the bigger the reward. Some levels end with a boss fight against Bowser, where the reward is freeing a Toad (or eventually, the Princess).

This is the **reward signal** in reinforcement learning. But here's the critical question: how do we actually **measure** how well the game was played? This is **evaluation**, or **reward modeling**, and it is where the machine learning engineering discipline really shines. (or research engineer, applied AI engineer.)

The evaluation could be the raw score. It could be the number of 1-ups gained. Coins collected. Goombas stomped. Time remaining. Different paths discovered. Most frequently, it is a **combination** of some or all of the above, weighted and balanced against each other. Do we reward Mario more for speed or for thoroughness? For survival or for aggression? For finding secrets or for staying on the critical path?

Evaluation Metric	Mario Measure	Agent Measure
Speed	Time remaining on the clock	Task completion latency
Score	Points accumulated	Overall output quality
Collection	Coins gathered	Information retrieved, resources used efficiently
Completeness	Hidden blocks found, secrets discovered	Edge cases handled, comprehensive coverage
Efficiency	Enemies defeated per life	Correct tool invocations per task
Exploration	Different paths taken	Novel approaches discovered

Designing these rewards is a strict machine learning engineering discipline. A poorly shaped reward function produces an agent that technically completes tasks but in degenerate ways, like a Mario speedrunner who clips through walls. Impressive, but not what we actually wanted. Reward hacking is the Goodhart's Law of agentic AI: when a measure becomes a target, it ceases to be a good measure.

Here's where the full picture comes together. How does Mario learn to complete levels? **Reinforcement learning.**

Mario starts each level knowing nothing about its specific layout. He has to:

1. **Observe** the current state, what's on screen, where the enemies are, what power-ups are available
2. **Decide** on an action based on his policy, jump, run, shoot, or wait
3. **Act** and receive feedback from the environment
4. **Update** his policy based on the outcome

This is the MDP (Markov Decision Process) loop. The same loop described in Agents Are Workflows. The same loop that every agentic AI system runs:

with Chromium. Filesystem snapshots are refreshed every thirty minutes by a cron, so a new session is working on a prompt within seconds. The lesson buried in that architecture is that the agent's productivity is bounded by the runtime's startup time, not by the model's tokens-per-second.

The browser-runtime category exists because driving Chromium is its own engineering problem — CDP, profiles, residential IPs, captcha handling, session persistence — and no general-purpose sandbox has it built in. Expect this to consolidate into the general-purpose vendors over the next eighteen months, the same way headless browsers absorbed into the major test frameworks.

It is also worth noting what nobody on this list does themselves. None of these vendors writes their own hypervisor. They all stand on top of the primitives in the previous section, which is what lets the category exist. The startups that have tried to build a complete stack — including the hypervisor — have ended up looking more like Cognition: years of engineering investment to get something that is finally indistinguishable from "we run on Firecracker with custom snapshot logic."

The Hyperscaler Sandbox Landscape

The hyperscalers got to this market late and are still catching up.

AWS Bedrock AgentCore ships a Code Interpreter and a Browser Tool as managed runtimes for agents. The isolation is microVM-class, the integrations point at the rest of the AWS data plane, and the pricing model is per-session. However, it has major gaps relative to the startups, where the runtime has to be coded and shipped like a Lambda function, instead of having the flexibility to support heterogeneous workloads, e.g, different agent harness + model combinations. Not a major problem for production, but a major problem for research and development.

Azure Container Apps Dynamic Sessions uses Hyper-V isolation and advertises sub-second start times for code interpreter sessions. It is the most directly comparable hyperscaler offering to E2B and Modal in terms of intent. The integration story is strongest if you are already on Azure OpenAI and AKS. It also suffers from the flexibility to support heterogeneous workloads.

GCP Cloud Run Sandboxes is ahead of the pack with the usability similar to Modal, E2B, Daytona and the likes. It's built on top of Cloud Run, which uses gVisor. Supports heterogeneous workloads.

The picks higher up the stack inherit these trade-offs. When a sandbox vendor advertises “VM-level isolation” they almost always mean Firecracker. When they advertise “millisecond cold starts” without VM isolation, they usually mean V8 isolates with a different threat model. The vendor brochure abstracts the primitive away; the threat model and the workload do not.

The Startup Sandbox Landscape

A category of “sandbox-as-a-service” companies has emerged in the last two years, almost all of them building on top of the primitives above. Northflank’s roundup of Modal Sandboxes alternatives is a good market snapshot — the differentiation is in the developer ergonomics, the snapshot model, and the mix of tools that come pre-wired, not in the isolation layer underneath.

Vendor	Isolation	Snapshot model	Cold start	Notable design choice
Modal	gVisor	Filesystem diffs from base image	Sub-second from snapshot	GPU support
E2B	Firecracker microVM	Full VM snapshot	~150ms	Open-source SDK, language-agnostic, popular in the agent dev community
Daytona	Containers / VMs	Forkable workspaces	Few seconds	Forked dev environments, OCI-compatible
Northflank	Containers, GPU-aware	Persistent volumes	Container-class	GPU sandboxes for agents that train or run inference inside the loop
Browserbase / Steel / Hyperbrowser	Containerized browsers	Session replay	Seconds	Browser-only runtimes for web agents — DOM, screenshots, CDP
Cloudflare Workers Sandbox	V8 isolates + containers	Object snapshots	Milliseconds	Edge-first, shared-nothing model
Vercel Sandbox	Firecracker	Snapshot from build	Sub-second	Tied to Vercel’s deploy and preview model

The reference Modal case study is Ramp Inspect: a background coding agent that now writes more than half of Ramp’s merged pull requests, with each session running in its own Modal sandbox containing Postgres, Redis, Temporal, RabbitMQ, a VS Code server, and a VNC stack

The value of Mario’s current state equals the expected immediate reward plus the discounted value of the next state. Should Mario jump NOW to get the coin, or wait and avoid the Goomba? The optimal policy balances immediate rewards against future outcomes.

Through repeated play (training episodes), Mario learns:

- Which obstacles can be jumped on vs. avoided
- When to use power-ups vs. save them
- Which pipes lead to shortcuts vs. dead ends
- How to handle boss fights

An agentic AI model goes through the same process. Through reinforcement learning (PPO, DPO, GRPO, or whatever the latest acronym is), the model learns:

- Which tools to invoke for which subtasks
- When to think longer vs. act immediately
- Which approaches work for which problem types
- How to decompose complex tasks into manageable steps

So who’s actually making all of this work? Mario doesn’t train himself.

To teach agent Mario to be really good at the game, we employ an **Machine Learning Engineer (MLE)**. The MLE is the game designer and coach rolled into one. They construct mock environments, set up the harnesses and tools, define the tasks that Mario needs to achieve, and most importantly, **design the reward function**. The MLE decides what “good” looks like. Do we reward Mario for speed? Thoroughness? Both? How much? This reward design is the single highest leverage decision in the entire pipeline. Get it right and Mario learns to play beautifully. Get it wrong and Mario learns to exploit glitches.

Once the MLE has designed the training setup, the **Machine Learning Systems Engineers (MLSys)** take over. They are the ones who actually run the show at scale. They set up the environment and agent Mario across hundreds to hundreds of thousands of environments, tasks and iterations. They manage the compute, the distributed training runs, the data pipelines. They collect the reasoning traces, the sequences of observations, actions, and outcomes from every single episode Mario plays. And from these traces, they run the reinforcement learning algorithms that allow agent Mario to learn from experience.

This is the unsexy but critical part. An MLE can design the most elegant reward function in the world, but without MLSys engineers standing up the infrastructure to run millions of training episodes and collect the resulting data, Mario never gets past World 1-1.

Worlds: Agent Frameworks and Domains

The Mushroom Kingdom isn't one level. It's organized into **Worlds**, each with a distinct theme and set of challenges:

World	Theme	Agent Domain
World 1	Grassland, basics	Simple text tasks, Q&A, summarization
World 2	Desert	Data analysis, structured environments
World 3	Water	Web and API navigation
World 4	Giant Land	Large-scale refactoring, migrations
World 5	Sky	Architecture and system design
World 6	Ice	Debugging in fragile or legacy environments
World 7	Pipe Maze	Complex multi-step workflows with dependencies
World 8	Bowser's Castle	Full autonomous task completion with adversarial conditions

Agent frameworks are the **World Maps** - they organize how the agent navigates between levels (subtasks) and worlds (domains). Some frameworks are rigid, you follow the path laid out. Others let you pick your route. The best ones (like how Super Mario Bros. 3 introduced the overworld map) let the agent make strategic choices about which level to tackle next.

But remember: the model is the product. The World Map (framework) is useful scaffolding, but Mario does the actual playing. As models get stronger, they need less scaffolding. Today's World 8 is tomorrow's World 1.

Conclusion

The entire agentic AI stack maps to Mario with surprising fidelity:

Mario	Agentic AI
Small Mario	Base pretrained model
Super Mushroom	Model harness and post-training
Power-ups	Agent tools and skills
Star Power	Engineering manager clearing blockers

gVisor	Userspace kernel intercepting syscalls; runc-compatible runtime	Container-class	Defense in depth where a microVM is overkill; GPU virtualization	Google Cloud Run, App Engine, Modal
Kata Containers	Lightweight VM per pod, OCI-compatible	Few hundred ms	Multi-tenant Kubernetes	Confidential Containers, some managed K8s
V8 isolates	Per-tenant JS heap inside a single process	Sub-millisecond	JavaScript-only, no arbitrary binaries	Cloudflare Workers, Deno Deploy

A few things are worth saying out loud about this table.

Containers are not a sandbox for agent code. They are a packaging and resource-control mechanism. The shared kernel means a kernel exploit, a misconfigured capability, or a sloppy seccomp profile takes down the isolation boundary. It is not okay for an agent that may execute attacker-controlled instructions hidden in a web page.

Firecracker is the de facto industry primitive for this category. AWS open-sourced it in 2018 to power Lambda and Fargate, and almost all agent-sandbox startup runs on top of it, including E2B, Fly.io, Vercel Sandbox. It boots a stripped-down Linux kernel inside KVM in roughly 125 milliseconds and uses about five megabytes of memory for the VMM itself. The combination of strong isolation, fast boot, high density is what makes thousands of concurrent rollouts economically viable.

gVisor is the middle path. It runs a userspace kernel that intercepts and re-implements Linux syscalls, giving you stronger isolation than namespaces without paying the full cost of a hardware VM. The trade-off is that some syscalls are slower or unsupported, which matters for workloads that hit the kernel hard. Google uses it for Cloud Run and App Engine; it is the right pick when you want defense-in-depth on top of containers but cannot move to microVMs.

Kata Containers fit a specific niche. They are an OCI-compatible runtime that wraps each pod in a lightweight VM, which lets you run untrusted workloads on Kubernetes without rewriting the orchestration layer. The cost is that you inherit Kubernetes' assumptions about pod lifetime, which do not match how agents actually behave.

V8 isolates are the wrong primitive for general agent workloads. They are extraordinary for JavaScript-only edge compute but an agent that needs to run arbitrary Python, install packages, drive a browser, or execute compiled binaries cannot live inside a V8 heap. Cloudflare's own Sandbox product addresses this by adding container-class compute alongside isolates.

Multi-tenancy at training scale. RL training spins up thousands of concurrent rollouts. Each rollout needs its own filesystem, its own process tree, its own network namespace. Without strong isolation primitives, one rollout's flaky shell command takes down a neighbor's training step.

Reproducibility. A sandbox you can snapshot is a sandbox you can replay. Replay is how you debug a six-hour agent trajectory without re-running the whole thing. It is also how you turn a production failure into a regression test.

A web app's runtime can be sloppy because the user is the one driving and a refresh fixes most things. An agent's runtime cannot, because the agent will keep going long after a human would have stopped to ask a question.

The Cognition team is direct about this in their post-mortem on building Devin's cloud agent infrastructure: containerized agents share a kernel, and a single compromised session can reach every other container's filesystem, credentials, and network connections. Because agents generate their own code, run arbitrary commands, and probe the environment in unpredictable ways, kernel-escape is not a theoretical risk — it is the working assumption. Their conclusion, after more than a year of hypervisor engineering, is the same one the broader infrastructure community has converged on for any untrusted-code workload: VM-level isolation, where each session gets its own kernel and there is no shared attack surface.

Manus team has famously built and snapped shotted all their agent runtimes in a restorable format so users can come back in the future to replay what agents have done.

The Isolation Primitive Stack

Most of the differentiation between sandbox vendors lives one layer down, in the isolation primitive they pick. There are five primitives in serious use, with very different trade-offs.

Primitive	Isolation model	Cold start	Workload fit	Notable users
Linux containers (runc, Podman)	Shared host kernel, namespaces + cgroups + seccomp	~100ms	Trusted code, internal CI	Docker, Kubernetes default
Firecracker	KVM-based microVM, dedicated kernel per VM, ~5MB Rust VMM	~125ms boot, sub-second from snapshot	Untrusted code at high density	AWS Lambda, Fargate, E2B, Fly.io

Levels	Task environments
Pipes, bricks, shells	Tool use within the environment
Objectives (speed, score, coins)	Task definitions
Flagpole / Boss	Reward signal and evaluation
Reward design	Reward modeling (ML engineering discipline)
Learning to play	Reinforcement learning
MLE as game designer	ML engineer designing tasks and rewards
MLSys running millions of episodes	ML infrastructure for training at scale
World Map	Agent framework
Multiplayer	Multi-agent systems

The next time someone asks you to explain agentic AI, ask them: have you played Mario?

If they can understand that Small Mario needs a mushroom to survive, power-ups to be effective, and practice to master each level, they can understand agentic AI development.

Now go save Princess Peach.

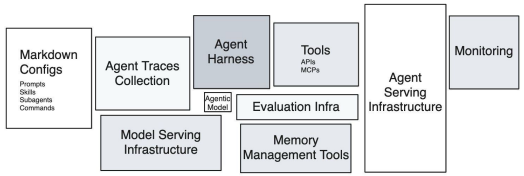
References

```
@article{
    leehanchung,
    author = {Lee, Hanchung},
    title = {It's-a Me, Agentic AI},
    year = {2026},
    month = {02},
    day = {18},
    howpublished = {\url{https://leehanchung.github.io}},
    url = {https://leehanchung.github.io/blogs/2026/02/18/mario-agent
}
```

Hidden Technical Debt of AI Systems: Agent Runtime

HAN, NOT SOLO · 27 APR 2026 · SOURCE

Eleven years ago, Sculley et al. drew the diagram everyone in MLOps has seen: a tiny black box labeled “ML Code” surrounded by a sprawl of much larger boxes — data collection, feature extraction, configuration, monitoring, serving infrastructure. The point of the diagram was that the model code is the smallest piece of a real ML system, and that everything else is where the technical debt accumulates.



Hidden Technical Debt in Agentic AI Systems

The same diagram is being redrawn for agents. The agentic model call is the small box. The largest box to the right that is currently driving most of the spend and influencing how system architecture will be done in the futureincidents, is the **agent runtime**, or agent serving infrastructure.

The agent is not the model. The agent is the harness plus the model, running inside the runtime. And almost nobody is treating it that way.

What an Agent Runtime Actually Is

A agentic model by itself maps from text input to text output. Other modalities are in play, but the primary fabric is still text. An agent is what you get when you wrap the agentic model with harnesses so it can take actions, observe effects, feed those observation back into the next call. The execution environment where that happens is the runtime.

Concretely, an agent runtime is the union of:

- **compute substrate** — a container, microVM, or full VM where code runs.
- **filesystem** the agent can read and write, often with snapshot and rollback semantics.
- **tools** — shell, code interpreter, browser, file editor, MCP servers — exposed to the model as callable interfaces.
- **network boundary** that defines what the agent can reach and what can reach it.
- **state model** that decides what persists across turns, across episodes, and across users.
- **lifecycle controller** that starts, suspends, snapshots, resumes, and tears down environments.

In the taxonomy of RL environments for LLM agents, these are the (harness) and (state) components. In production, this is the part that decides whether your agent finishes a task in eight seconds or eight minutes, whether two users can share an environment safely, and whether a malicious prompt can read your secrets.

Most teams shipping agents today have not built this layer. They have rented it, glued it together from cloud primitives that were designed for a different workload, or simply not thought about it yet. That last group is the one accumulating the most debt.

Why Sandboxing Is Not Optional

Models hallucinate code. They run `rm -rf`. They paste credentials into curl commands. They follow instructions embedded in untrusted documents. They retry the same failing command in a tight loop and burn through a quota. None of these are exotic edge cases — they are the modal behavior of capable models when handed unrestricted tool access.

The sandbox is what stops these behaviors from becoming incidents. Four reasons it has to exist as a first-class layer rather than a hopeful disclaimer in the system prompt:

Isolation against the model’s mistakes. A coding agent that mounts your repo and has shell access can, and eventually will, delete the wrong directory. Filesystem isolation, copy-on-write snapshots, and per-session ephemerality turn destructive actions into recoverable ones.

Isolation against prompt injection. The agent reads tool outputs. Tool outputs are attacker-controlled the moment the agent visits a webpage, opens a PDF, or processes a customer email. A sandbox is the only thing standing between an injected instruction and your production database. This is the agent-systems version of the SQL injection lesson, and we are at roughly the 2003 stage of learning it.